UNIVERSITÀ
DEGLI STUDI
DI PADOVA

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

INFORMATION ENGINEERING DEPARTMENT

MASTER'S DEGREE IN COMPUTER ENGINEERING

APEROL from Networks: Analyzing Pipeline and Embedding Representations for Optimized Learning (from Networks)

Professor
Fabio Vandin

Students
Marco Annunziata, 2160851
Silvia Mondin, 2141201
Sveva Turola, 2160852

ACADEMIC YEAR 2025-2026

Contents

1 Introduction

Graphs have emerged as a natural model for the representation and analysis of data and complex systems in various domains. For instance, link prediction in social networks can help to understand the spreading process of rumors or epidemics [1], and in biological networks, it is commonly used for predicting novel interactions between proteins [2].
However, most traditional machine learning algorithms are not designed to work directly with graph-structured data, as they require numeric vectors or matrices as inputs. To address this challenge, embedding techniques have been developed to learn low-dimensional vector representations of nodes, edges, or entire graphs while preserving their topological properties.
In order to design an effective embedding technique, it is necessary to consider several criteria [3]. Two of these are adaptability and scalability. An embedding method is considered adaptable if it can be utilized for various data and tasks. The scalability of an embedding technique is determined by its capacity to process large-scale networks within a reasonable amount of time.

The objective of this study is to evaluate known embedding methods from the perspective of these two criteria.
From the perspective of adaptability, the objective is to determine the potential impact of the graph domain on the precision of an embedding method. In other words, the objective is to ascertain whether there exist approaches that are better suited for a particular field, whether there is one embedding technique that consistently outperforms the others, or if the methods are comparable.
In the context of scalability, the objective is to identify the embedding methods that are better suited to process large graphs. This is accomplished by evaluating the tradeoff between the accuracy of the predictions and the time required for training and inference.

2 Embedding methods

Node embedding is the task of mapping nodes into an embedding space, where each node v is represented by a vector $\mathbf{z_v} \in \mathbb{R}^d$ using the mapping function $ENC : v \to \mathbf{z}_v$. The encoding of nodes into embeddings has to be such that similarity between each pair of nodes is kept in the correspondent embeddings. In order to achieve this, a similarity score $S(u, v)$ is computed between pairwise nodes and a decoder function $DEC : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ is used for computing a similarity score for the embeddings. The discrepancy between the two scores is then compared and minimized by means of a loss function, such as the empirical loss:

$$\mathcal{L} = \sum_{i,j} \ell(DEC(\mathbf{z}_u, \mathbf{z}_v), S(u, v))$$

The node embedding methods are defined by the characteristics of the encoder and decoder.

In this project, four of these methods are considered:

- **Node2Vec** [4] A shallow encoding method and essentially a tunable version of the DeepWalk algorithm, which is often used in the scientific literature as the baseline for comparing node embedding methods.
- **LINE** [5] A shallow encoding which is an highly-scalable approximation algorithm.
- **Deep Variational Network Embedding in Wasserstein Space** (DVNE) [6] A deep encoding method using a Variational Autoencoder. This method has a particular approach of encoding each node to a probability distribution, so in this case the similarity score between embeddings is a distance between probability distributions, which can be measured using the Wasserstein distance.
- **GraphSage** [7] A deep encoding method making use of a GNN that aggregates the features of each node's neighborhood and learns to generate node embeddings from that.

3 Link Prediction

In this study, the main task considered is link prediction. The link prediction task is defined as the estimation of the existence of a link between two nodes in a network, based on the observed links. Formally,

Network	$\|V\|$	$\|E\|$	Directed	Weighted
Pennsylvania [11, 12]	1,088,092	3,083,796	✓	✗
Padua (province) [13]	122,728	304,184	✓	✓
Hong Kong (city) [14]	43,620	91,542	✓	✓
Italian Covid-19 Retweet Network [15]	221,574	2,332,721	✓	✓
Deezer [16, 17]	54,573	846,915	✗	✗
GitHub Developers [18, 19]	37,700	289,003	✗	✗
Mus Musculus Protein Interactions [20] [1]	20,969	808,988	✗	✓
Saccharomyces cerevisiae Protein Interactions [20] [2]	5,786	103,831	✗	✓
Bio-grid-fission-yeast [21]	2,026	12,637	✗	✗

Table 1: Datasets used in the project

given a graph $G = (V, E)$, where V is the set of nodes and E the set of edges, the link prediction task aims to predict whether an edge $(u, v) \in (V \times V) \setminus E$ exists or not, based on the observed edges in E.

In order to perform link prediction using node embeddings, a common approach is to compute a feature vector for each pair of nodes (u, v) based on their embeddings $\mathbf{z}_u$ and $\mathbf{z}_v$. Subsequently, the feature vectors are utilized as input to a binary classifier that predicts the presence or absence of a link between the two nodes.

It is important to note that the quality of the predictions is potentially dependent on two factors: the embeddings themselves and the classifier utilized. To this end, classifiers of varying complexity have been considered:

- **Support Vector Machines** (SVM) [8] chosen as linear baseline.
- **Random Forest** (RF) [9] chosen for its fast computation.
- **Multilayer Perceptron** (MLP) [10] chosen for its accurate predictions.

4 Datasets

The datasets used in this project are nine and they are divided into three categories: road networks, social networks, and biological networks. For each of these categories, three datasets of increasing size were selected: one small, one medium, and one large.

The datasets, with their main properties (i.e., the number of nodes, the number of edges, if it is directed, and if it is weighted), are presented in Table 1.

5 Experiments

In this section, the experimental setup used to evaluate the performance of the selected graph embedding methods for the link prediction task is described. The various steps of the pipeline are illustrated, including dataset preprocessing, data splitting, metrics used for evaluation, implementation details of the embedding methods and classifiers, and hardware specifications.
The code used for the experiments is available in the GitHub repository of the project [22].

5.1 Pipeline

The link prediction task requires a sample of edges from the graph (positive edges) and a sample of edges from the complement graph (negative edges, in this case, do not include self-loops). Consequently, each dataset is partitioned as follows.
Consider graph $G = (V, E)$, where V denotes the set of vertices and E is the set of edges. A first split of

[1] Confidence score of > 0.4
[2] Confidence score of > 0.7

the set of edges E is needed for the embedding algorithms, creating E_{embed} and E_{pred}.
E_{embed} is used for the subgraph $G_{\text{embed}} = (V, E_{\text{embed}})$ given as input to the embedding methods: the shallow embedding methods use it to learn a lookup table Z of node embeddings, whereas the deep embedding methods aim to learn a function f that can generalize well.
The remaining edges in E_{pred} are instead reserved for the link-prediction task, where the second split occurs: from E_{pred} the sets E_{train}, E_{val}, E_{test} are created for the training, validation and test of the prediction model. More precisely, E_{train} is used to train the prediction models, E_{val} to select one of them. After that, the selected model is retrained using both E_{train} and E_{val}. Finally, E_{test} is used to evaluate its performance.

In order to achieve a balanced distribution of negative edges, each set contains an equal number of positive and negative examples. Furthermore, in order to avoid data leakage, the sampled negative edges must be distinct from those used internally by the embedding algorithms during training.

The input of the prediction models are edge embeddings. For each edge (u, v), its corresponding embedded representation is constructed by concatenating the embeddings z_u, z_v of its incident nodes.

The split used in this study is: (from E) 80% for E_{embed} and 20% for E_{pred}; (from E_{pred}) 60% for E_{train}, 20% for E_{val} and 20% for E_{test}.

5.2 Dataset preprocessing

A preprocessing pipeline is utilized in which all datasets are converted into CSV files and only the useful features are extracted, if present.

Initially, all the datasets were manually converted to CSV files, as some of them were not directly available in this format. Subsequently, a Python script was implemented to automate the following preprocessing of the datasets. Undirected edges (u, v) are represented using a pair of directed edges (u, v) and (v, u). Unweighted datasets are represented as having a uniform weight of 1.0 for all edges. Finally, the node IDs are mapped to numeric consecutive IDs, with the sequence beginning at 0.

In regard to the negative edges, it was necessary to associate them with weights. To achieve this, the weights of the negative edges were sampled from the weights of the positive edges. This process was implemented to simulate the distribution of the weights of the positive edges.

5.3 Metrics

Two metrics that are generally used for link prediction are the Area Under the Receiver Operating Characteristic curve (AUROC), and the Area Under the Precision-Recall curve (AUPR). AUROC is historically considered the primary performance metric used to evaluate the performances of link prediction methods [23]. However, AUROC favors accurate classification of positive examples, at the cost of misclassifying the negative ones. In a scenario like the link prediction problem, which is inherently biased towards the negative class, this approach may not be optimal and can overestimate the performance of the methods. Consequently, the AUPR curve was also selected, as it can provide better evaluation of link prediction in the presence of class imbalance.

The values returned by both metrics range from 0 to 1. The AUROC with a value of 1 is indicative of perfect classification, while 0.5 represents random guesses. Similarly, an AUPR value of 1 indicates perfect precision and recall, whereas a value equal to the ratio of positive samples to the total number of samples corresponds to random predictions; in this case, we consider balanced datasets, so the random baseline is 0.5.

For the validation phase, the AUROC metric is used to select the best model, while for the final evaluation both AUROC and AUPR are reported.

5.4 Embedding and classifiers implementation

Before describing the specific implementations used for the embedding techniques and classifiers, it is important to point out that all implementations have been adapted to run on a GPU.

In regard to the implementation of the embedding techniques mentioned in Sec 2, the following implementations have been adopted and modified as needed.

- **Node2Vec** [24, 25] The implementation used the following hyperparameters: walk length = 20, context size for positive samples = 10, walks per node = 10, number of negative samples for each positive sample = 1, p = 1.0, q = 1.0.
- **LINE** [26] The original implementation was optimized for computational efficiency and training stability (for example, using sparse embeddings, SparseAdam optimizer and Batch Matrix Multiplication). In addition, a linear learning rate decay was integrated into the training process. At last, the output generation was modified. In fact, the embeddings for first-order and second-order proximity were trained separately, concatenated into a single representation, and then L2-normalized.
- **DVNE** [27] The implementation has been severely rewritten for adapting it to our link prediction pipeline and to make it more similar to the original paper. There are also some changes which resulted in the model being more robust, in particular: the use of log-scaled wasserstein distances, the manual computation of features for the nodes during the training process and using z-score for their standardization. Also a fast approximated method of picking negatives is used for picking the triplets needed in the training process. In the original paper, DVNE was only tested on undirected graphs, while in this work it has been applied to both directed and undirected graphs without any specific adaptation.
- **GraphSage** [28] The implementation was modified to support a Graph object as input instead of the original text-based format. In particular, two functions were integrated to generate node features and training labels directly from the graph topology. In the end, the training process was adjusted to return the trained model.

In regard to the implementation of the classifiers mentioned in Sec 3, the following implementations have been adopted.

- **SVM** Model with regularization parameter 0.01, trained using hinge loss and L2 regularization.
- **MLP** Simple model using 1 layer with 16 hidden channels, cross entropy loss and Adam optimizer. In the case of this particular classifier, the decision was made to employ the validation set for implementing early stopping. Consequently, the model does not go through retraining on the union of the training and validation sets after the validation process.
- **RF** XGBRFClassifier [29]. This model integrates the Random Forest algorithm within the XGBoost framework, utilizing the histogram-based tree method (hist) to accelerate training on the GPU.

5.5 Hardware specifications

The experiments have been conducted using the DEI cluster *"Blade"*. The specifications for the nodes of the cluster are available in the open documentation [30].
The experiments utilized node gpu[2], which comprises 32 CPU cores (2x Intel Xeon Gold 5218 CPU @ 2.30/3.90GHz), 1.5TB RAM, 8 GPUs Nvidia RTX 3090. The whole pipeline run for two hours using one GPU, two CPUs per task and 5.44 GB of memory.

6 Results

Figure 2 illustrates the training times for each embedding method across the various datasets.
As expected, Node2Vec requires the longest training time, particularly for larger datasets, due to its reliance on random walks. It exhibits training times that are an order of magnitude greater than those of the other methods.
On the other hand, DVNE demonstrates the fastest training times.
As is illustrated, the time required for training is not uniform. This phenomenon may be attributed to the influence of two factors: the size of the graph and its internal structure.
In fact, Node2vec has been observed to record peaks for social networks (GitHub Developers) and valleys for biological networks (Mus Musculus Protein Interaction).
In contrast, LINE appears to exhibit peaks on biological networks, including those of the Mus Musculus

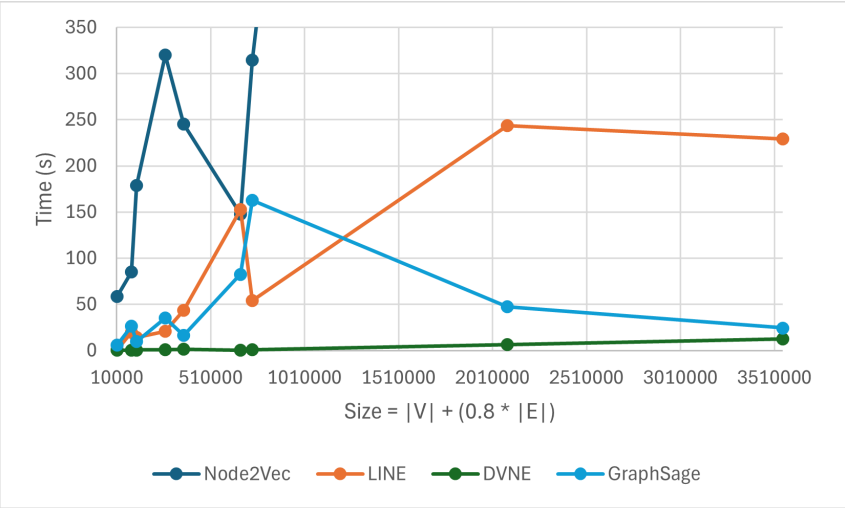

Figure 2: Training time comparison. The x-axis represents the datasets through their size $|V| + 0.8 * |E|$, which approximates the total number of nodes and edges used during training.

Protein Interaction and Saccharomyces cerevisiae Protein Interaction.
GraphSage reveals a notable peak for Deezer and valleys for road networks.

Table 2 reports the AUROC and AUPR values of the best performing classifier for each embedding technique across all datasets.
It can be seen that Node2Vec outperforms the other embedding techniques across the majority of the datasets; however DVNE shows competitive results, surpassing Node2Vec in datasets such as Hong Kong and Italian Covid-19 Retweet Network.
Instead, LINE proves to be the worst performing embedding method in almost all scenarios, probably due to a low number of epochs for the training (necessary to maintain a feasible trade-off between performance and computational time/memory complexity) or its limitation in capturing only local (first and second order) proximity, which may fail to represent the global structure of complex and sparse networks.
GraphSage and DVNE demonstrate a stable performance, each of them showing consistency across all categories of datasets. This suggests that these methods are more robust to variations in graph topology.
Two notable extremes are the Italian Covid-19 Retweet Network and Deezer. For the first one, all the embedding methods achieve high values of AUPR and AUROC (with the lowest value of AUROC being 0,9202 for GraphSage), indicating a clear community network where links are easily predictable; while for the second one, all embedding techniques show low values, especially LINE, which reaches an AUPR value of 0,3704 (the lowest value of the entire table), likely due to the high noise or sparse connectivity of the graph.
For what concerns classifiers, RF reaches high performance across most embedding techniques and dataset types. Specifically, it is the best performing model for Node2Vec on road networks and for GraphSage in almost all scenarios. On the other hand, MLP remains highly effective when paired with DVNE and with Node2Vec on social networks.

A complete overview of the training times and results can be found in the GitHub repository of the project [22].

7 Conclusions

Graph embedding addresses the fundamental challenge of representing complex relational data, structured as graphs or networks, in a format suitable for machine learning. Traditional machine learning models require fixed-size numerical vectors as input, but graphs are inherently non-Euclidean. The core problem is how to map nodes into a low-dimensional continuous vector space while preserving essential structural and semantic properties.

Dataset	Node2Vec			LINE		
	Best Model	AUROC	AUPR	Best Model	AUROC	AUPR
Pennsylvania	RF	**0,8369**	**0,7717**	MLP	0,5770	0,5840
Padua	RF	**0,8973**	**0,8551**	RF	0,5805	0,5492
Hong Kong	RF	0,8278	0,7649	MLP	0,6157	0,6219
Italian Covid-19 Retweet Network	MLP	0,9596	0,9666	MLP	0,9513	0,9575
Deezer	MLP	**0,7191**	**0,5652**	RF	0,5415	0,3704
GitHub Developers	MLP	**0,9225**	**0,8865**	RF	0,6932	0,5578
Mus Musculus Protein Interaction	RF	**0,8213**	**0,6877**	RF	0,7500	0,6121
Saccharomyces cerevisiae Protein Interaction	RF	**0,8805**	**0,8130**	RF	0,8005	0,7153
Bio-grid-fission-yeast	MLP	**0,8754**	**0,7962**	MLP	0,7773	0,6829

Dataset	DVNE			GraphSage		
	Best Model	AUROC	AUPR	Best Model	AUROC	AUPR
Pennsylvania	MLP	0,7959	0,7710	RF	0,6958	0,6286
Padua	RF	0,7761	0,7121	RF	0,8083	0,7343
Hong Kong	MLP	**0,8694**	**0,8464**	RF	0,8365	0,7643
Italian Covid-19 Retweet Network	MLP	**0,9716**	**0,9756**	RF	0,9202	0,8808
Deezer	MLP	0,6888	0,5480	RF	0,5769	0,3986
GitHub Developers	MLP	0,9063	0,8540	MLP	0,7239	0,4947
Mus Musculus Protein Interaction	RF	0,8091	0,6649	RF	0,7517	0,5762
Saccharomyces cerevisiae Protein Interaction	RF	0,8601	0,7491	RF	0,8411	0,7177
Bio-grid-fission-yeast	RF	0,8636	0,7541	MLP	0,8622	0,7764

Table 2: AUROC and AUPR values for the embedding methods. For each dataset, the best results are highlighted in bold.

This study has presented an empirical evaluation of four graph embedding methods (Node2Vec, LINE, DVNE, and GraphSage) for the task of link prediction guided by two principal criteria: adaptability (performance across different graph types) and scalability (computational efficiency). This study utilizes a systematic experimentation approach on nine distinct datasets, encompassing road, social, and biological networks.

Adaptability was examined in relation to performance on datasets. A general absence of statistically significant influence was detected with respect to the type of graph; however, performance appears to be contingent upon the specific dataset under consideration. Consequently, it would be worthwhile to examine the topologies of these datasets to ascertain whether a correlation exists.
Furthermore, even if the Node2Vec embedding method was identified as the most effective approach, it is important to note that the absence of substantial node features in both DVNE and GraphSage may have limited their performance. It would be worthwhile to ascertain whether there are any significant features that can be adapted to multiple datasets.

The scalability of the embedding methods has been examined in terms of the time required for training. A general trend indicates that the training times for all embedding methods increase with the size of the datasets. However, the internal structure of the graphs appears to play a significant role.

It has been confirmed that no single embedding method is universally superior. Node2Vec excels when resources are available, whereas DVNE and GraphSage offer better scalability with only a modest sacrifice in performance, making them preferable for larger graphs or time-sensitive applications.

References

[1] Romualdo Pastor-Satorras et al. "Epidemic processes in complex networks". In: *Reviews of modern physics* 87.3 (2015), pp. 925–979.

[2] Björn H Junker and Falk Schreiber. *Analysis of biological networks*. Vol. 2. Wiley Online Library, 2008.

[3] Anthony Baptista et al. "Zoo guide to network embedding". In: *Journal of Physics: Complexity* 4.4 (2023), p. 042001.

[4] Aditya Grover and Jure Leskovec. *node2vec: Scalable Feature Learning for Networks*. 2016. arXiv: 1607.00653 [cs.SI]. URL: https://arxiv.org/abs/1607.00653.

[5] Jian Tang et al. "LINE: Large-scale Information Network Embedding". In: *Proceedings of the 24th International Conference on World Wide Web*. WWW '15. International World Wide Web Conferences Steering Committee, May 2015, pp. 1067–1077. DOI: 10.1145/2736277.2741093. URL: http://dx.doi.org/10.1145/2736277.2741093.

[6] Dingyuan Zhu et al. "Deep Variational Network Embedding in Wasserstein Space". In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '18. London, United Kingdom: Association for Computing Machinery, 2018, pp. 2827–2836. ISBN: 9781450355520. DOI: 10.1145/3219819.3220052. URL: https://doi.org/10.1145/3219819.3220052.

[7] William L. Hamilton, Rex Ying, and Jure Leskovec. *Inductive Representation Learning on Large Graphs*. 2018. arXiv: 1706.02216 [cs.SI]. URL: https://arxiv.org/abs/1706.02216.

[8] Corinna Cortes and Vladimir Vapnik. "Support-vector networks". In: *Machine learning* 20.3 (1995), pp. 273–297.

[9] Leo Breiman. "Random forests". In: *Machine learning* 45.1 (2001), pp. 5–32.

[10] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. "Learning representations by back-propagating errors". In: *Nature* 323.6088 (1986), pp. 533–536.

[11] Jure Leskovec et al. "Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters". In: *Internet Mathematics* 6.1 (2009), pp. 29–123.

[12] Jure Leskovec and Andrej Krevl. *SNAP Datasets: Stanford Large Network Dataset Collection*. https://snap.stanford.edu/data/roadNet-PA.html.

[13] Marco Annunziata. *Padua_Network_dataset_2025*. 2025. URL: https://github.com/Remdox/Padua_Network_dataset_2025.

[14] yzengal. *RoadNetwork-China-City*. 2021. URL: https://github.com/yzengal/RoadNetwork-China-City/blob/main/Hongkong.road-d.tar.gz.

[15] V. Mesina et al. *Italian Covid-19 Retweet Network (2020-2022) [Data set]*. 13th International Conference on Complex Networks and Their Applications (CNA2024), Istanbul, Turkey. Zenodo. 2024. URL: https://doi.org/10.5281/zenodo.13909011.

[16] Benedek Rozemberczki et al. "GEMSEC: Graph Embedding with Self Clustering". In: *Proceedings of the 2019 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining 2019*. ACM. 2019, pp. 65–72.

[17] Jure Leskovec and Andrej Krevl. *SNAP Datasets: Stanford Large Network Dataset Collection*. https://snap.stanford.edu/data/gemsec-Deezer.html.

[18] Benedek Rozemberczki, Carl Allen, and Rik Sarkar. *Multi-scale Attributed Node Embedding*. 2019. arXiv: 1909.13021 [cs.LG].

[19] Jure Leskovec and Andrej Krevl. *SNAP Datasets: Stanford Large Network Dataset Collection*. http://snap.stanford.edu/data/github-social.html.

[20] Damian Szklarczyk et al. "The STRING database in 2023: protein–protein association networks and functional enrichment analyses for any sequenced genome of interest". In: *Nucleic Acids Research* 51.D1 (2023), pp. D638–D646. DOI: 10.1093/nar/gkac1000. URL: https://string-db.org/cgi/download.

[21] Ryan A. Rossi and Nesreen K. Ahmed. "The Network Data Repository with Interactive Graph Analytics and Visualization". In: *AAAI*. 2015. URL: https://networkrepository.com.

[22] Turola Sveva Annunziata Marco Mondin Silvia. 2026. URL: https://github.com/Remdox/LFN_Graph_Embeddings.

[23] I. Bhargavi Kalyani, A. Rama Prasad Mathi, and Niladri Sett. "Evaluating link prediction: new perspectives and recommendations". In: *International Journal of Data Science and Analytics* 20.7 (2025), pp. 6855–6886. ISSN: 2364-4168. DOI: 10.1007/s41060-025-00858-0. URL: https://doi.org/10.1007/s41060-025-00858-0.

[24] Matthias Fey and Jan E. Lenssen. "Fast Graph Representation Learning with PyTorch Geometric". In: *ICLR Workshop on Representation Learning on Graphs and Manifolds*. 2019.

[25] Matthias Fey et al. "PyG 2.0: Scalable Learning on Real World Graphs". In: *Temporal Graph Learning Workshop @ KDD*. 2025.

[26] dmpierre. 2019. URL: https://github.com/dmpierre/LINE.

[27] Lakshya-99. 2020. URL: https://github.com/Lakshya-99/Deep_Variational_Network_Embedding/tree/master.

[28] William L. Hamilton et al. 2018. URL: https://github.com/williamleif/graphsage-simple/.

[29] Tianqi Chen and Carlos Guestrin. "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. ACM, Aug. 2016, pp. 785–794. DOI: 10.1145/2939672.2939785. URL: http://dx.doi.org/10.1145/2939672.2939785.

[30] Università degli Studi di Padova. *Cluster "Blade"*. URL: https://docs.dei.unipd.it/en/CLUSTER/Overview.

Contribution of Authors and AI Usage

The contributions for this project are:

- Code
  - dataset_preprocessing: Silvia Mondin
  - dataset_utils: Silvia Mondin
  - pipeline_utils: Marco Annunziata
  - pipeline: Marco Annunziata
  - utils: Silvia Mondin
  - README: Marco Annunziata
  - General optimization: Marco Annunziata
- Report
  - Introduction: Silvia Mondin
  - Embedding methods: Marco Annunziata
  - Link Prediction: Silvia Mondin
  - Datasets: Sveva Turola
  - Experiments: Silvia Mondin
  - Pipeline: Marco Annunziata
  - Dataset preprocessing: Silvia Mondin
  - Metrics: Sveva Turola
  - Hardware specifications: Marco Annunziata
  - Results: Silvia Mondin, Sveva Turola
  - Conclusions: Silvia Mondin
- Embedding methods (code + report)
  - Node2vec: Silvia Mondin
  - LINE: Sveva Turola
  - DVNE: Marco Annunziata
  - GraphSage: Sveva Turola
- Classifiers (code + report)
  - SVM: Silvia Mondin
  - MLP: Marco Annunziata
  - RF: Sveva Turola

Each group member has been assigned to specific parts of the project. Even so, a collaborative effort was made to the overall design of the project, resulting in minor contributions across the entire codebase and reports.

The following AI tools were used:

- Gemini and Copilot were used for
  - suggestions on the code: torch metric library, adapting input graphs, node features for DVNE and GraphSage.
  - optimization of the code.
  - fixing bugs.
- Copilot was used to suggest report snippets.
- DeepL was used to correct the grammar and syntax of the text.